Software Design and Architecture: Short Notes with Examples

1. Software Architecture

Definition: High-level structure of a software system showing components and their relationships.

Example: Designing a social media app where backend servers, databases, and front-end apps are organized.

2. Monolithic Architecture

Definition: Entire application built as a single unit.

Example: A blog website where login, article management, and comments are all handled in one big project.

3. Layered Software Architecture

Definition: Organizes code into layers (Presentation, Business Logic, Data Access).

Example: A banking app where the UI (Presentation Layer) calls a Service Layer (Business Logic) which accesses the Database Layer.

4. Tiered Software Architecture

Definition: Physical separation of layers into different servers.

Example: A travel booking site with a frontend server (UI), a backend server (processing), and a separate database server.

5. Microservices Software Architecture

Definition: Breaks down application into small, independent services.

Example: Netflix has different microservices for authentication, recommendations, and video streaming.

6. Serverless Architecture

Definition: Developers deploy code without managing servers; backend runs automatically.

Example: A photo-sharing app where uploading a photo triggers an AWS Lambda function to resize images.

7. Event Driven Architecture

Definition: Components communicate by producing and consuming events.

Example: In an e-commerce site, when a payment is made, it triggers an event that updates inventory and sends confirmation emails.

8. Software Design

Definition: The process of defining how software should be built and how components interact.

Example: Designing an online library system where each book, member, and issue request is a separate class.

9. SOLID Principles

Definition: Five principles to create maintainable and scalable software:

- Single Responsibility A class should have only one one job. example: Bad: A User class that handles user data and logs user activity. Good: Separate User class and Logger class.
- Open/Closed This principle states that “Software entities (classes, modules, functions, etc.) should be open for extension, but closed for modification” which means you should be able to extend a class behavior, without modifying it.
- Liskov Substitution This principle ensures that any class that is the child of a parent class should be usable in place of its parent without any unexpected behaviour. example-One of the classic examples of this principle is a rectangle having four sides. A rectangle’s height can be any value and width can be any value. A square is a rectangle with equal width and height. So we can say that we can extend the properties of the rectangle class into square class.
- Interface Segregation Clients should not be forced to depend on interfaces they do not use.
- Dependency Inversion High-level modules should not depend on low-level modules. Both should depend on abstractions.Meaning:Instead of a class directly depending on another concrete class, it should depend on an interface or abstract class. example: Use interfaces for database access (IDatabase) so the high-level service doesn’t care whether it's using MySQL or MongoDB.

Example: A "ReportGenerator" class only generates reports and does not handle file storage (Single Responsibility).

10. Software Design Principles

Definition: Guidelines for good software design like DRY (Don't Repeat Yourself) and KISS (Keep It Simple, Stupid).

KISS (Keep It Simple, Stupid):Simplicity should be a key goal. Avoid unnecessary complexity.
DRY (Don't Repeat Yourself):Avoid duplicating code. Every piece of knowledge should have a single, unambiguous representation in the system.

Example: Avoid writing the same validation logic repeatedly; instead, use a single function.

11. Software Design Patterns

Definition: Reusable solutions to common design problems.

Example: Using the Factory Pattern to create different types of notification senders (Email, SMS, Push) based on user choice.

12. 4+1 Architectural View Model

Definition: Represents architecture using 5 views:

- Logical View (functional requirements) The logical view is concerned with the system's functionality as it pertains to end-users
- Development View (software management) How the system is implemented
  Focus: Software modules/components and organization in development
- Process View (dynamic aspects) The process view focuses on the system's run-time behavior
- Physical View (hardware mapping) Focus: Mapping software to hardware nodes
- Scenarios (+1) (example flows)

Example: In an online payment system, the Logical View shows the payment workflow, the Development View shows the project structure, the Process View shows transaction handling, the Physical View shows server deployment, and Scenarios show customer payment flows.

software architecture: High-level structure of the software system. but software design are Detailed design of system components and their interactions.